SAECOIN — DEVELOPMENT SETUP GUIDE

Document: Development_Setup.md
Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Status: Planning
Primary OS: Windows 11 64-bit
Available RAM: 16 GB

---

1. PURPOSE

This document explains how to prepare a Windows 11 computer for SAECOIN development.

The development environment will support:

- Rust blockchain development
- Full node development
- CPU mining development
- Future GPU mining development
- Android wallet development
- Windows desktop wallet development
- Blockchain explorer development
- Automated testing
- GitHub collaboration

The environment will be configured gradually to avoid unnecessary software installation and excessive resource consumption.

---

2. DEVELOPMENT STRATEGY

SAECOIN development will use a phased installation strategy.

Phase 1 — Essential Tools

Install only:

1. Git
2. Rust
3. Visual Studio Build Tools, if required
4. Visual Studio Code or another suitable code editor

These tools are sufficient to begin the initial Rust core development.

Phase 2 — Blockchain Development

Prepare:

- Rust workspace
- Cargo configuration
- Formatting tools
- Testing tools
- Database dependencies

Phase 3 — Android Wallet

Install:

- Android Studio
- Android SDK
- JDK
- Android Emulator

Phase 4 — Desktop Wallet and Explorer

Install:

- Node.js
- TypeScript
- Frontend dependencies
- Tauri prerequisites

Phase 5 — GPU Mining

Install GPU-specific development tools only after selecting the mining algorithm and GPU implementation strategy.

---

3. INSTALL GIT

Git will manage the SAECOIN source code and project history.

Download Git for Windows from:

https://git-scm.com/downloads/win

Recommended installation:

- Use Git from the command line and third-party software.
- Use the default Git editor or select a preferred editor.
- Enable Git Credential Manager.

- Use the recommended line-ending settings.

Verify installation:

git --version

Expected result:

A Git version number should appear.

---

4. INSTALL RUST

Rust will be the primary programming language for the blockchain core.

Official installation page:

https://www.rust-lang.org/tools/install

For Windows, use the official Rust installation method.

Install the stable toolchain.

After installation, open a new PowerShell window.

Verify:

rustc --version
cargo --version
rustup --version

All three commands should return version information.

---

5. INSTALL RUST DEVELOPMENT COMPONENTS

Install the standard formatting and linting components:

rustup component add rustfmt
rustup component add clippy

Verify:

cargo fmt --version
cargo clippy --version

These tools will help maintain consistent code quality.

---

6. WINDOWS BUILD TOOLS

Some Rust dependencies require native Windows compilation support.

If compilation reports that a C/C++ compiler or Windows SDK is missing, install the required components through:

https://visualstudio.microsoft.com/visual-cpp-build-tools/

Typical components may include:

- MSVC C++ build tools
- Windows SDK
- MSVC linker

The exact required components depend on the Rust target and dependencies.

Do not install the full Visual Studio IDE unless it is needed.

---

7. INSTALL A CODE EDITOR

A lightweight editor is recommended for the 16 GB RAM development computer.

One option is:

https://code.visualstudio.com/

Recommended extensions:

- rust-analyzer
- Even Better TOML
- GitLens
- Error Lens

Extensions are optional and may be changed later.

The editor should provide:

- Rust syntax highlighting
- Code completion
- Error diagnostics
- Git integration

- Terminal access

---

8. VERIFY THE RUST ENVIRONMENT

After installing the required tools, open PowerShell and run:

```
rustc --version
cargo --version
rustup show
git --version
```

Then run:

```
cargo new saecoin-test
cd saecoin-test
cargo check
cargo test
```

Expected result:

- A basic Rust project is created.
- Cargo successfully checks the project.
- The default test suite passes.

This is only an environment verification test, not the SAECOIN blockchain implementation.

The temporary test project can be removed after verification.

---

9. DEVELOPMENT DIRECTORY

Use a dedicated directory for SAECOIN.

Suggested structure:

```
SAECOIN/
    docs/
    crates/
    apps/
    services/
    tests/
    scripts/
    configs/
    .github/
    Cargo.toml
```

rust-toolchain.toml
README.md
LICENSE
.gitignore

This is a proposed structure.

The exact Rust workspace members must be decided according to which directories contain Rust packages.

Non-Rust applications must not be added as Cargo workspace members.

---

10. RUST TOOLCHAIN PINNING

Once the Rust version is selected for the project, create:

rust-toolchain.toml

Example structure:

```
[toolchain]
channel = "stable"
components = ["rustfmt", "clippy"]
profile = "minimal"
```

For production development, the team should select and pin a tested toolchain version rather than allowing uncontrolled toolchain changes.

The example above uses the stable channel and is not a final version lock.

---

11. ANDROID DEVELOPMENT SETUP

Android development will be required for the SAECOIN mobile wallet.

Install Android Studio:

https://developer.android.com/studio

Required components will include:

- Android SDK
- Android SDK Platform
- Android SDK Build-Tools
- Android Emulator

- JDK compatible with the selected Android Gradle Plugin

The Android wallet will use:

- Kotlin
- Jetpack Compose

Android development can be installed later, after the blockchain core and wallet architecture are sufficiently defined.

---

12. WINDOWS DESKTOP WALLET SETUP

The proposed Windows desktop wallet technology is:

- Rust
- Tauri
- TypeScript

The Tauri development environment may require:

- Rust toolchain
- Node.js
- Windows build tools
- WebView2 Runtime
- Tauri-specific dependencies

Official documentation:

https://v2.tauri.app/start/prerequisites/

Install these dependencies only when desktop wallet development begins.

---

13. EXPLORER DEVELOPMENT SETUP

The proposed explorer frontend will use:

- React
- TypeScript
- Node.js

Official Node.js website:

https://nodejs.org/

A package manager will be selected during frontend project setup.

The explorer backend technology remains subject to the final architecture decision.

---

14. DATABASE SETUP

RocksDB is a proposed initial database option for blockchain storage.

Before installation or integration:

1. Confirm the Rust database library.
2. Confirm Windows compatibility.
3. Benchmark read/write performance.
4. Test database recovery.
5. Test crash consistency.
6. Verify storage requirements.

Database installation must follow the selected library's official instructions.

Do not finalize the database choice before benchmarking and reviewing compatibility.

---

15. GPU MINING DEVELOPMENT SETUP

GPU mining tools should not be installed yet.

The required software depends on:

- Final PoW algorithm
- GPU programming model
- Supported GPU vendors
- Windows driver requirements
- Rust integration strategy

Potential technologies may include CUDA or OpenCL, but neither is selected as the final implementation.

GPU support must be designed and benchmarked after the PoW algorithm is finalized.

---

16. MEMORY AND PERFORMANCE MANAGEMENT

The development computer has 16 GB RAM.

Recommended practices:

- Keep only necessary applications open.
- Avoid running multiple Android emulators simultaneously.
- Build one major component at a time.
- Prefer SSD storage.
- Close unused development tools.
- Use incremental compilation.
- Avoid running a full node, explorer, emulator, and multiple build processes simultaneously unless resources permit.

The exact memory requirements will be measured during development.

---

17. GITHUB SETUP

Create a GitHub account if one is not already available.

Official website:

https://github.com/

Create a private repository during early development.

Suggested repository name:

saecoin

Initial Git commands after creating the project directory:

git init
git branch -M main

Configure the remote repository after creating it on GitHub.

Never upload:

- Private keys
- Seed phrases
- Wallet databases
- Passwords
- API secrets
- Release signing keys

A suitable ".gitignore" file must be created before committing development files.

---

18. ENVIRONMENT VERIFICATION CHECKLIST

Before SAECOIN coding begins, verify:

- [ ] Windows 11 is updated.
- [ ] Git is installed.
- [ ] Rust is installed.
- [ ] Cargo is working.
- [ ] rustfmt is installed.
- [ ] Clippy is installed.
- [ ] C/C++ build tools are available if required.
- [ ] Code editor is installed.
- [ ] A temporary Rust project builds successfully.
- [ ] A temporary Rust test passes.
- [ ] SAECOIN repository is created.
- [ ] Documentation is stored safely.

Optional tools should be installed only when their development phase begins.

---

19. IMPORTANT SECURITY RULES

1. Download development tools from official websites.
2. Do not execute unknown scripts from repositories.
3. Review third-party dependencies before adoption.
4. Do not store wallet secrets in source code.
5. Never share private keys or seed phrases.
6. Keep development, testnet, and mainnet data separate.
7. Use Git commits to preserve development history.
8. Back up important project documentation.
9. Review AI-generated code before accepting it.
10. Never treat a successful build as proof of security.

---

20. FINAL SETUP POLICY

The SAECOIN development environment will be prepared gradually.

Initial development should focus on Rust and the blockchain core.

Android, desktop, explorer, and GPU-specific tools can be added when their development phases begin.

The 16 GB RAM computer will remain the primary development machine during the initial stages.

If project requirements exceed its practical capacity, additional hardware or a separate build server may be considered later.

DOCUMENT STATUS

Document: Development_Setup.md
Version: 1.0
Project: SAECOIN
Operating System: Windows 11
Available RAM: 16 GB
Status: Planning
Implementation Status: Not Started
Consensus Parameters: Provisional